

▼ Estrutura de Controle - Loops

Durante a execução de um programa, é muito comum surgir a necessidade de repetir um mesmo trecho de código várias vezes, seja por uma quantidade fixa de repetições ou de forma dinâmica, enquanto uma determinada condição for satisfeita. Para lidar com esse tipo de situação, utilizamos as estruturas de repetição, também conhecidas como loops.

Em Python, loops são estruturas de controle que permitem executar repetidamente um bloco de código, de acordo com uma condição ou por um número definido de iterações. Eles são fundamentais para automatizar tarefas repetitivas, percorrer listas de dados, processar entradas do usuário e controlar fluxos de execução mais complexos.

Nesta aula, vamos conhecer os principais loops da linguagem Python, entender como funcionam e aprender quando utilizar cada um deles.

▼ FOR

A estrutura de repetição `for` é utilizada quando queremos executar um bloco de código ao longo de um conjunto de valores, definido por um objeto iterável, como listas, tuplas, strings ou intervalos numéricos.

Em outras palavras, o `for` permite que o programa repita uma tarefa para cada elemento de um iterável, sem a necessidade de controlar manualmente o número de repetições.

A cada iteração, a variável de controle assume o próximo valor do iterável, e o bloco de código associado ao `for` é executado.

Neste momento, ainda não vimos em detalhes estruturas como listas e tuplas, que também são objetos iteráveis muito comuns em Python. Por enquanto, utilizaremos o `for` principalmente com intervalos de valores. Mais adiante no curso, quando estudarmos listas e tuplas, retornaremos ao uso do `for` para percorrer essas estruturas de forma mais completa e prática.

```
# Usando for para repetir um bloco de código um número específico de vezes
for i in range(10):
    print(i)
```

```
0
1
2
3
4
5
6
7
8
9
```

```
# Usando for para iterar sobre os caracteres de uma string
for letra in "Python":
    print(letra)
```

```
P
y
t
h
o
n
```

```
# Multiplicar um número por 2, dez vezes
numero = 1
for _ in range(10):
    numero *= 2
    print(numero)
```

```
2
4
8
16
32
64
128
256
512
1024
```

Adiantando um pouco do conceito de listas, no `for` podemos passar uma sequência de variáveis as quais queremos iterar, utilizando os "[" e "]"

```
for num in [-1, 2, 4, -17, 25]:  
    print(i)
```

```
25  
25  
25  
25  
25
```

```
for professor in ["Zezé", "Tim", "Toms"]:  
    print(professor)
```

```
Zezé  
Tim  
Toms
```

▼ WHILE

A estrutura de repetição `while` (enquanto) é utilizada quando desejamos repetir um bloco de código enquanto uma determinada condição for verdadeira. Diferentemente do `for`, em que o número de repetições geralmente é conhecido previamente, o `while` é mais indicado para situações em que não sabemos exatamente quantas vezes o código deverá ser executado, pois a repetição depende do resultado de uma condição lógica. O laço continua sendo executado enquanto a condição for avaliada como `True` e é interrompido assim que essa condição se torna `False`.

```
a = 0  
b = 10  
  
while a < b:  
    a += 1  
    print("Valor de A: ", a)
```

```
Valor de A: 1  
Valor de A: 2  
Valor de A: 3  
Valor de A: 4  
Valor de A: 5  
Valor de A: 6  
Valor de A: 7  
Valor de A: 8  
Valor de A: 9  
Valor de A: 10
```

```
a = 0  
b = 10  
c = 0  
  
while a < b or c != 5:  
    a += 1  
    c += 0.5  
  
print("Valor de A", a)  
print("Valor de C", c)
```

```
Valor de A 10  
Valor de C 5.0
```

```
a = 0  
b = 15  
c = 0  
  
while a < b and c != 5:  
    a += 1  
    c += 0.5  
  
print("Valor de A", a)  
print("Valor de C", c)
```

```
Valor de A 10  
Valor de C 5.0
```

▼ Instruções de controle de fluxo (Loop control statements)

Durante um processo iterativo seja usando `for` or `while` é comum precisas controlar o loop, seja encerra-lo ou pular a iteração atual.

- `continue`: Encerra a iteração atual e pula direto pra próxima.
- `break`: Encerra o processo de loop.

- `pass`: Instrução nula, não faz nada. Só ocupa espaço de indentação.

```
a = 10
for i in [-2, -1, 0, 1, 2]:
    print(a/i)
```

```
-5.0
-10.0
```

```
-----
ZeroDivisionError                                Traceback (most recent call last)
Cell In[21], line 3
      1 a = 10
      2 for i in [-2, -1, 0, 1, 2]:
----> 3     print(a/i)

ZeroDivisionError: division by zero
```

```
a = 10
for i in [-2, -1, 0, 1, 2]:

    if i == 0:
        continue # pula a iteração

    print(a/i)
```

```
-5.0
-10.0
10.0
5.0
```

No python não existe um Do While como em C++, logo se deseja fazer um código parecido com ele precisa utilizar o `break`

```
a = 0
b = 10

while True: # Loop infinito

    a += 1

    condicao = (a > b)

    if condicao:
        break

print("Valor de A:", a)
```

```
Valor de A: 11
```

```
# Podemos utilizar o break para adicionar mais condições em um for
a = 10
for i in range(10):
    a /= 10

    if a < 1e6:
        print("Essa condicao foi aceita primeira")
        break
```

```
Essa condicao foi aceita primeira
```

```
for professor in ["Toms", "Tim", "Zezé", "Raimundo"]:
    if professor == "Zezé":
        print("Encontrei o Zé")
        break
    else:
        print(f"Professor {professor} não é o Zé")
```

```
Professor Toms não é o Zé
Professor Tim não é o Zé
Encontrei o Zé
```

Sobre o `pass`

Em Python, os blocos de código são definidos por indentação. Isso significa que estruturas como `for`, `if`, `while`, `def`, etc. exigem ao menos uma instrução dentro do bloco.

Se você deixar o bloco vazio, o código quebra com erro de sintaxe (`IndentationError` ou `SyntaxError`).

Veja o exemplo abaixo:

```
for i in range(10):  
    # fazer codigo dps  
  
if a > 0:  
    a = -9
```

Esse código gera erro porque o `for` espera um bloco indentado válido, e comentários não contam como instruções.

Para esses casos, usamos o `pass`:

```
for i in range(10):  
    # fazer codigo dps  
    pass  
  
if a > 0:  
    a = -9
```

O `pass` é uma instrução nula (no-op):

- Não executa nenhuma ação
- Existe apenas para satisfazer a exigência sintática de um bloco indentado

Exemplos da Aula

```
# Método de Newton para encontrar Raiz Quadrada de 2 aproximada  
  
x = 1.0          # chute inicial  
tol = 1e-6       # tolerância  
max_iter = 100   # número máximo de iterações  
  
iteracao = 0  
  
while abs(x**2 - 2) > tol and iteracao < max_iter:  
    x = x - (x**2 - 2) / (2*x)  
    iteracao += 1  
  
print("Raiz aproximada:", x)  
print("Número de iterações:", iteracao)
```

```
Raiz aproximada: 1.4142135623746899  
Número de iterações: 4
```

```
# Uma simulação ao longo do tempo  
  
t_max = 5 # seg  
dt = 0.005 # seg  
  
num_iter = int(t_max/dt)  
  
# Condições iniciais  
x = 0  
dx = 0  
ddx = 0.5  
  
# Simulacao  
for i in range(num_iter):  
  
    # calculando novo estado  
    dx_new = dx + ddx * dt  
    x_new = x + dx_new * dt  
  
    if i % 100 == 0: # Só printar a cada 1000 iterações  
        print(f"x: {x_new:.2f}, dx: {dx_new:.2f}")  
  
    # Atualizando estado  
    x = x_new  
    dx = dx_new
```

```
x: 0.00, dx: 0.00  
x: 0.06, dx: 0.25  
x: 0.25, dx: 0.50  
x: 0.57, dx: 0.75  
x: 1.01, dx: 1.00  
x: 1.57, dx: 1.25  
x: 2.26, dx: 1.50  
x: 3.08, dx: 1.75  
x: 4.02, dx: 2.00  
x: 5.08, dx: 2.25
```